North Human Solutions

Évaluation Technique — Intelligence Artificielle

NIVEAU JUNIOR

48 heures · Assistant FAQ E-Commerce avec RAG

Candidat(e)	Date de remise

Lien GitHub (repo privé partagé avec le recruteur)

1. Vue d'ensemble du test

🎯 Contexte du projet
Tu rejoins l'équipe IA de North Human Solutions pour une mission réelle.
Un client e-commerce veut un assistant FAQ intelligent capable de répondre aux
questions de ses clients à partir de sa documentation produit et de ses politiques
de service. Ton rôle : concevoir, construire et livrer ce pipeline de bout en bout.

Paramètre	Valeur
Durée totale	48 heures à partir de la réception du document
Format	Take-home — travail en autonomie complète
Niveau visé	Junior IA (0–2 ans d'expérience)
Stack	Libre — tout choix doit être justifié dans le README
Modèle LLM	OpenAI, Mistral, Cohere, Ollama local, HuggingFace...
Livraison	Repo GitHub privé + courriel de remise

2. Sujet — Assistant FAQ E-Commerce

2.1 Mise en situation

Le client ShopVite (nom fictif) possède une boutique en ligne de produits électroniques. Ses agents de support reçoivent quotidiennement les mêmes questions de la part des clients : délais de livraison, politique de retour, garantie, comparaison de produits.

Ta mission est de construire un assistant IA qui peut répondre à ces questions automatiquement, en s'appuyant exclusivement sur les documents fournis — sans inventer d'informations.

2.2 Architecture attendue

Voici l'architecture minimale que le pipeline doit respecter :

1 **Ingestion des documents**

Charger les fichiers fournis (PDF / texte / JSON), les découper en chunks cohérents.

2 **Vectorisation**

Générer les embeddings de chaque chunk et les indexer dans un vector store persistant.

3 **Retrieval contextuel**

À partir d'une requête utilisateur, retrouver les k chunks les plus pertinents.

4 **Génération augmentée**

Injecter les chunks dans le contexte du LLM et générer une réponse fidèle aux sources.

5 **API & Exposition**

Exposer le pipeline via une API REST avec un endpoint clair et un Dockerfile.

2.3 Spécification de l'API

L'endpoint principal attendu est le suivant :

```
POST /ask

Content-Type: application/json

Body: { "question": "Quelle est la politique de retour ?" }


Response: {

  "answer": "La politique de retour est de 30 jours pour tout produit...",

  "sources": ["faq_retours.pdf, section 3", "conditions_generales.txt"],
```

```
  "confidence": "high"
}
```

Tu dois également implémenter :

→ GET /health → retourne le statut de l'API (200 OK ou message d'erreur)
→ Un message d'erreur clair si la question est hors contexte des documents

2.4 Prompt Engineering

Le system prompt est une composante critique de ce test. Il doit remplir les fonctions suivantes :

→ Donner une identité et un ton précis à l'assistant (professionnel, concis, en français)
→ Forcer l'assistant à répondre uniquement à partir du contexte fourni
→ Produire un refus poli et utile pour les questions hors-scope
→ Citer la source utilisée dans chaque réponse
→ Implémenter au moins une technique avancée : few-shot examples, chain-of-thought, ou garde-fous explicites

Dans ton fichier reflection.md, explique en 5 à 10 lignes pourquoi tu as fait ces choix de prompting.

3. Sources de données — Datasets recommandés

Tu as deux options : utiliser les datasets publics ci-dessous, ou créer ton propre corpus de documents (3 à 5 fichiers minimum). Dans les deux cas, le corpus doit être pertinent pour un contexte e-commerce / support client.

3.1 Datasets publics recommandés pour ce sujet

Nom	Source	Description	Usage IA	Taille
Amazon QA	Hugging Face	Questions-réponses produits Amazon (~500k Q/A)	RAG / Retrieval	~1.2 GB
E-Commerce FAQ	Kaggle	FAQ génériques e-commerce : retours, livraison, paiement	RAG baseline	~15 MB
FinanceBench	Hugging Face	Docs financiers structurés — bonne pratique chunking	Chunking / RAG	~300 MB
SQuAD 2.0	Hugging Face	Wikipedia Q/A — idéal pour tester le retrieval	Éval retrieval	~35 MB
Natural Questions	Hugging Face / TF	Questions naturelles Google — diversité des requêtes	Éval générale	~40 MB

BEIR Benchmark	GitHub / HF	Suite complète d'évaluation retrieval (18 datasets)	Éval RAG	Variable

3.2 Où trouver ces datasets

Plateforme / Outil	URL	Pourquoi l'utiliser
Hugging Face Datasets	*huggingface.co/datasets*	Bibliothèque principale — chercher 'e-commerce', 'FAQ', 'QA'
Kaggle Datasets	*kaggle.com/datasets*	Nombreux datasets e-commerce et support client en CSV/JSON
Google Dataset Search	*datasetsearch.research.google.com*	Moteur de recherche de datasets du monde entier
Papers With Code	*paperswithcode.com/datasets*	Datasets de recherche NLP avec benchmarks associés
CommonCrawl	*commoncrawl.org*	Corpus web massif — données brutes pour fine-tuning
MTEB Benchmark	*huggingface.co/mteb*	Benchmark embeddings — utile pour choisir ton modèle d'embedding

3.3 Option : créer ton propre corpus

Si tu préfères créer ton propre corpus de documents pour ShopVite (recommandé pour montrer de l'initiative), tu peux utiliser les outils suivants :

→ Scraper des FAQs publiques de boutiques réelles (Amazon, Best Buy, Fnac)
→ Générer des documents fictifs cohérents avec GPT / Claude (politique retour, guide produit, CGV)
→ Combiner 3 à 5 fichiers de formats différents (PDF + TXT + JSON ou Markdown)

Peu importe l'origine des données — ce qui est évalué, c'est la qualité de ton pipeline RAG, pas la richesse du corpus.

4. Livrables attendus

Fichier / Dossier	Description détaillée et critères de qualité
README.md (requis)	Architecture du pipeline (schéma ASCII ou image) · Instructions de setup en moins de 3 commandes · Choix techniques justifiés pour chaque composant · Exemples de requêtes avec résultats attendus
src/ (requis)	Code source organisé en modules logiques · Commentaires sur les fonctions non triviales · Gestion des erreurs sur tous les appels externes (LLM, API, vector store)

Déploiement (requis — au choix)	Option A — Docker : image buildable (docker build + docker run), variables documentées dans .env.example · Option B — HuggingFace Spaces : déploiement Gradio ou Streamlit sur huggingface.co/spaces, lien public partagé dans le README · Les deux options sont évaluées de façon équivalente
reflection.md (requis)	Justification du prompt engineering (5-10 lignes) · Ce que tu ferais différemment avec plus de temps · Une limitation identifiée de ton système actuel
data/ (requis)	Corpus de documents utilisés (ou script de téléchargement si trop volumineux) · Fichier data_sources.md listant l'origine de chaque document
demo.mp4 ou notebook (requis)	Démonstration du pipeline sur 3 requêtes minimum · Au moins 1 question dans le scope, 1 question hors-scope · Durée : 2 à 5 minutes
eval/ (bonus)	Évaluation quantitative sur 10 questions minimum · Métriques : faithfulness + answer relevancy (RAGAS ou custom) · Tableau de résultats commenté

5. Grille d'évaluation

Critère d'évaluation	Description	Points
Pipeline RAG fonctionnel de bout en bout	Ingestion → chunking → embedding → retrieval → génération	**30 pts**
Qualité du prompt engineering	Robustesse, anti-hallucination, gestion hors-scope, citation sources	**20 pts**
Qualité du code	Structure, lisibilité, DRY, gestion d'erreurs, commentaires	**20 pts**
Déploiement (Docker ou HF Spaces)	Application accessible et fonctionnelle, README clair, choix documenté	**15 pts**
Communication technique	reflection.md, justification des choix, documentation data	**10 pts**
Évaluation quantitative (bonus)	Métriques RAGAS ou équivalent sur 10+ questions	**5 pts (bonus)**
TOTAL		**100 pts**

6. Ressources pour démarrer

6.1 Documentation officielle

Plateforme / Outil	URL	Pourquoi l'utiliser

LangChain Docs	*python.langchain.com/docs*	Guides RAG, agents, vectorstores, prompts
LlamaIndex Docs	*docs.llamaindex.ai*	Alternative à LangChain, très bonne pour le RAG
RAGAS Docs	*docs.ragas.io*	Évaluation RAG : faithfulness, context recall, etc.
ChromaDB Docs	*docs.trychroma.com*	Vector store local simple à configurer
FastAPI Docs	*fastapi.tiangolo.com*	API REST Python, auto-documentation Swagger
HF Spaces Docs	*huggingface.co/docs/hub/spaces*	Déploiement Gradio / Streamlit gratuit — alternative au Dockerfile

6.2 Tutorials de référence

→ LangChain RAG Tutorial officiel : python.langchain.com/docs/tutorials/rag/
→ DeepLearning.AI — Building RAG applications with LangChain (cours gratuit)
→ Towards Data Science — RAG from scratch (article de référence)
→ YouTube : James Briggs — Complete RAG pipeline in 30 minutes

6.3 Contact et questions

Pour toute question concernant les contraintes techniques ou les documents fournis, contacte-nous à l'adresse indiquée dans le courriel de remise. Les questions portant sur les choix technologiques ne recevront pas de réponse — c'est précisément ce que nous évaluons.